Singleton Design Pattern

1. What is the Singleton Design Pattern?

Definition

Singleton is a design pattern that ensures a class has **only one instance** during program execution and provides a **global access point** to that instance.

In simple terms, Singleton controls object creation so that multiple objects of the same class cannot exist. It is useful when one shared object must coordinate access to a common resource or maintain a consistent global state.

2. Problem Addressed by Singleton

In many systems, allowing multiple objects of a class may cause:

- Inconsistent data
- Resource overuse
- Conflicting system states

Key Idea

Singleton solves this problem by allowing exactly one shared object to manage system-wide actions, ensuring consistency and avoiding duplicate resource usage.

3. When to Use Singleton

Singleton is suitable when:

- Exactly one object is required
- The object must be globally accessible
- Centralized control is necessary

Typical examples:

- Database connection manager
- Configuration manager
- Logging service
- Cache manager
- Resource manager

4. Core Implementation Principles

Important Points

A Singleton class usually contains:

- **Private constructor** – prevents direct object creation using `new`
- **Static instance variable** – stores the single object
- **Public access method** – returns the instance

5. Basic Singleton (Single-Threaded)

```
public class Singleton {

    private static Singleton instance;

    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }

    private Singleton() { }
}
```

Limitation: This implementation is **not thread-safe**. In a multithreaded environment, multiple objects may still be created.

6. Issue in Multi-Threaded Environment

If two threads call `getInstance()` at the same time:

- Both may find `instance == null`
- Both may create separate objects
- The Singleton guarantee is broken

Warning

This problem is called a **race condition**.

7. Thread-Safe Singleton using Synchronization

```
public class Singleton {

    private static Singleton instance;

```

```
    public static synchronized Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }

    private Singleton() { }
}
```

Advantages:

- Thread-safe
- Supports lazy initialization

Disadvantage:

- Synchronization adds performance overhead

8. Eager Initialization Singleton

```
public class Singleton {

    private static final Singleton INSTANCE = new Singleton();

    public static Singleton getInstance() {
        return INSTANCE;
    }

    private Singleton() { }
}
```

Characteristics:

- Object is created at class loading time
- Thread-safe without synchronization
- May waste memory if the object is never used

9. Double-Checked Locking (Lazy + Efficient)

This approach keeps lazy initialization and reduces synchronization cost by locking only when the instance is being created.

```
public class Singleton {

    private static volatile Singleton instance;

    public static Singleton getInstance() {
        if (instance == null) {
```

```
            synchronized (Singleton.class) {
                if (instance == null) {
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }

    private Singleton() { }
}
```

`volatile` ensures proper visibility of the initialized object across threads.

10. Bill Pugh Singleton (Static Inner Class)

This technique uses a static inner helper class. The instance is created only when `getInstance()` is called for the first time.

```
class Singleton {
    private Singleton() {}

    private static class Holder {
        private static final Singleton instance = new Singleton()
            ;
    }

    public static Singleton getInstance() {
        return Holder.instance;
    }
}
```

Advantages:

- Lazy initialization
- Thread-safe
- No synchronization overhead

Disadvantage:

- Slightly less intuitive for beginners

11. Breaking Singleton using Reflection

Java Reflection can access private constructors and create multiple objects.

```
private Singleton() {
    if (INSTANCE != null)
        throw new IllegalStateException("Instance already exists"
            );
}
```

This defensive check helps prevent reflection-based attacks by stopping the creation of a second object.

12. Enum-Based Singleton

```
public enum Singleton {
    INSTANCE;

    public void showMessage() {
        System.out.println("Hello Singleton");
    }
}
```

Usage:

```
Singleton.INSTANCE.showMessage();
```

Key Idea

Enum-based Singleton is considered the **most robust Singleton implementation in Java** because it is naturally thread-safe and protects against serialization and reflection issues.

Advantages:

- Simplest and safest
- Thread-safe by default
- Safe against serialization problems
- Safe against reflection attacks

Disadvantage:

- Less flexible because enums cannot extend classes

13. Example: Database Connection Singleton

```
class DatabaseConnection {
    private static DatabaseConnection instance;

    private DatabaseConnection() {
        System.out.println("Connecting to database...");
    }

    public static DatabaseConnection getInstance() {
        if (instance == null) {
            instance = new DatabaseConnection();
        }
```

```
        return instance;
    }

    public void query(String sql) {
        System.out.println("Executing: " + sql);
    }
}
```

Usage:

```
public class Main {
    public static void main(String[] args) {
        DatabaseConnection db1 = DatabaseConnection.getInstance()
            ;
        DatabaseConnection db2 = DatabaseConnection.getInstance()
            ;

        System.out.println(db1 == db2); // true
        db1.query("SELECT * FROM users");
    }
}
```

Here, both `db1` and `db2` refer to the same object, confirming that only one instance exists.

14. When NOT to Use Singleton

Avoid Singleton when:

- Multiple instances are actually needed
- Unit testing requires mocking or dependency injection
- Global state leads to tight coupling

15. Comparison of Singleton Approaches

Approach	Thread-Safe	Lazy	Secure	Performance	Complexity	Recommended
Basic	No	Yes	No	High	Low	No
Synchronized	Yes	Yes	No	Low	Low	Rare
Eager	Yes	No	Partial	High	Low	Sometimes
Double-Checked Locking	Yes	Yes	Partial	High	High	Sometimes
Bill Pugh	Yes	Yes	Partial	High	Medium	Yes
Enum	Yes	No	Yes	High	Low	Yes

16. Conclusion

Singleton restricts object creation to a single instance. It is useful when one object must coordinate access to shared resources or maintain a single global state.

In Java:

- **Enum-based Singleton** is the safest and simplest overall
- **Bill Pugh Singleton** is often the best choice when lazy initialization is needed without synchronization overhead

Quick Revision Points

Exam Revision Box

- Singleton ensures **only one object** of a class.
- It provides a **global access point**.
- Basic Singleton is **not thread-safe**.
- Synchronized Singleton is safe but slower.
- Eager initialization is simple and thread-safe.
- Double-checked locking improves efficiency.
- Bill Pugh Singleton is lazy and thread-safe.
- Enum Singleton is the **most robust** in Java.
- Avoid Singleton if it causes **tight coupling** or hurts testability.